

PRACTICAL ACTIVITY 1: Solve 1 classical-crypto CTF challenge (Caesar/Vigenère-based).

The image shows the 'caesar' challenge page on the picoCTF 2019 website. The challenge is categorized as 'Medium' and 'Cryptography'. The description asks to decrypt a message. The message is 'message'. The page shows that 57,260 users solved the challenge, with a 64% like rate. The flag format is picoCTF{crossingtherubiconvfhsgdv}.

Below the challenge page is a terminal window showing the solving process:

```
REIAHNEVIL-picoctf@webshell:~$ wget https://challenge-files.picoctf.net/c_fickle_tempest/e1502af9ec6ede316e796b88ed49d0ed1be4e7724511e3e96a2f42e6af597f01/data.enc
--2026-03-10 23:58:28-- https://challenge-files.picoctf.net/c_fickle_tempest/e1502af9ec6ede316e796b88ed49d0ed1be4e7724511e3e96a2f42e6af597f01/data.enc
Resolving challenge-files.picoctf.net (challenge-files.picoctf.net)... 3.160.5.64, 3.160.5.95, 3.160.5.18, ...
Connecting to challenge-files.picoctf.net (challenge-files.picoctf.net)[3.160.5.64]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 36 [application/octet-stream]
Saving to: 'data.enc'

data.enc
100%[=====] 36 --.-KB/s in 0s

2026-03-10 23:58:28 (34.3 MB/s) - 'data.enc' saved [36/36]

REIAHNEVIL-picoctf@webshell:~$ ls
README.txt data.enc
REIAHNEVIL-picoctf@webshell:~$ ./data.enc
-bash: ./data.enc: Permission denied
REIAHNEVIL-picoctf@webshell:~$ ^C
REIAHNEVIL-picoctf@webshell:~$ file data.enc
data.enc: ASCII text
REIAHNEVIL-picoctf@webshell:~$ cat data.enc
picoCTF{hwtxxnslmjwzgnhtswlvhsgdv}
```

Solving Process:

Since the challenge title in picoCTF 2019 already indicated that the encryption method involved a **Caesar cipher**, I approached the problem knowing that the ciphertext had likely been shifted by a fixed number of positions in the alphabet. After downloading the provided file `data.enc`, I first used the `file` command to determine the file type and confirmed that it was ASCII text. I then displayed its contents using the `cat` command, which revealed the string `picoCTF{hwtxxnslmjwzgnhtswlvhsgdv}`. Although the text already followed the picoCTF flag format, the characters inside the braces appeared to be encrypted. Because a Caesar cipher uses a single shift value as its key, I attempted a brute-force approach by trying all possible shifts of the alphabet. I gradually shifted the ciphertext backward one position at a time and checked whether the result produced a meaningful English phrase. After testing multiple possibilities, I found that a shift of **21** produced readable text: `crossingtherubiconvfhsgdv`. Placing this decrypted text back into the flag format revealed the final flag: `picoCTF{crossingtherubiconvfhsgdv}`.

Reflection:

This challenge reinforced my understanding of how simple classical encryption techniques can be broken through systematic analysis. Since the challenge title already indicated that it involved a Caesar cipher, I immediately knew that the encryption used a single shift applied consistently across the entire message. Instead of trying to guess the key directly, I applied a brute-force approach by testing different shift values until the ciphertext produced readable text. By doing this, I was able to observe how small changes in the shift value significantly altered the output, and how quickly the correct plaintext could be identified once the proper key was reached.

Working through the challenge also highlighted the limitations of the Caesar cipher as a secure encryption method. Because there are only a small number of possible shifts in the alphabet, it becomes very easy to break the cipher by simply testing each possible key. This exercise demonstrated why classical ciphers like Caesar are no longer suitable for protecting sensitive information in modern systems. Overall, the activity helped me practice using basic command-line tools, applying brute-force techniques in cryptanalysis, and recognizing patterns in encrypted text to determine the correct decryption.